PROYECTO OSCILOSCOPIO + GENERADOR · STM32

Cómo está hecha la app

Guía para exponer la parte del código: qué hace cada archivo `.py`, cómo se conectan entre sí y cómo viaja un dato desde la placa hasta la pantalla.



La app funcionando (pestaña Osciloscopio). CH1: senoidal de 1 kHz; CH2: cuadrada de 3 kHz con ciclo útil del 25 %. Captura tomada con la placa simulada que usamos para las pruebas.

---

1. La idea en una página
2. Las piezas y cómo se conectan
3. Cómo viaja un dato
4. Archivo por archivo
5. Los otros archivos del proyecto
6. Ideas para contar y preguntas probables

Versión de la app: 0.9.0 beta · 13 archivos de Python, unas 7500 líneas · Python 3.10 o más nuevo.

1. La idea en una página

El instrumento tiene dos programas que trabajan juntos:

El firmware (en la placa)

Está escrito en C y corre en el STM32. Hace lo que tiene que ser **rápido y exacto en el tiempo**: tomar muestras con el ADC a 1,6 millones por segundo, sacar la forma de onda por el DAC, contar flancos para el frecuencímetro, generar el PWM y manejar el módulo AD9833.

La app (en la PC)

Está escrita en Python. Hace todo lo demás: la **ventana** con los controles, los **gráficos**, las **cuentas** (pasar códigos a volts, mediciones, FFT, Bode, ajuste de polos y ceros), guardar la **configuración** y las **calibraciones**, y armar los **informes**.

Se comunican por el **USB de la placa**, que en la PC aparece como un puerto COM. La app manda **comandos de texto** (por ejemplo `CAP 2000 1000 200000 1 2048 0 20`, «capturá 2000 puntos a 200 kS/s disparando en CH1») y la placa contesta con texto (`OK ...`) o, en el caso del osciloscopio, con un encabezado de texto seguido de los datos en binario.

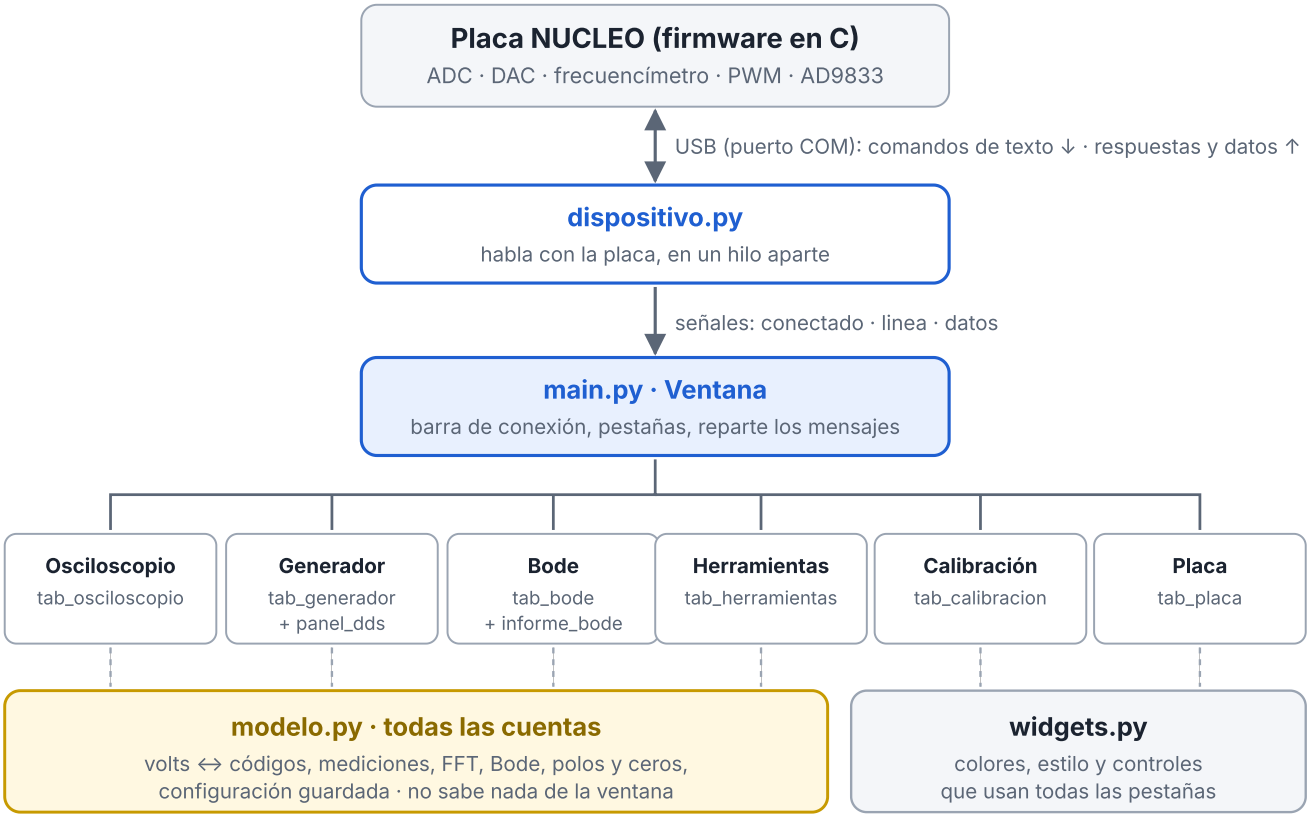
Las bibliotecas que usa

Biblioteca	Para qué
PySide6 (Qt)	La ventana, los botones, las pestañas, los diálogos, los hilos y las «señales» para avisar cosas entre partes. También se usa para dibujar el PDF del Bode.
pyqtgraph	Los gráficos que se actualizan rápido (osciloscopio, FFT y Bode, decenas de cuadros por segundo).
numpy	Todas las cuentas con vectores: pasar 8192 muestras a volts de una vez, FFT, ajustes por mínimos cuadrados.
pyserial	Abrir el puerto COM y leer y escribir bytes.

La regla de oro del diseño

Los cálculos están separados de la interfaz. Todo lo matemático vive en `modelo.py`, que no sabe nada de ventanas ni botones. Las pestañas (`tab_*.py`) solo leen los controles, le piden las cuentas a `modelo.py` y muestran el resultado. Gracias a eso las cuentas se pueden probar solas, con más de cien pruebas automáticas, sin abrir la app ni conectar la placa.

2. Las piezas y cómo se conectan



Cada caja es un archivo. Las flechas muestran por dónde pasan los mensajes de la placa. Todas las pestañas usan `modelo.py` para las cuentas y `widgets.py` para verse igual.

Tres conceptos que aparecen en todo el código

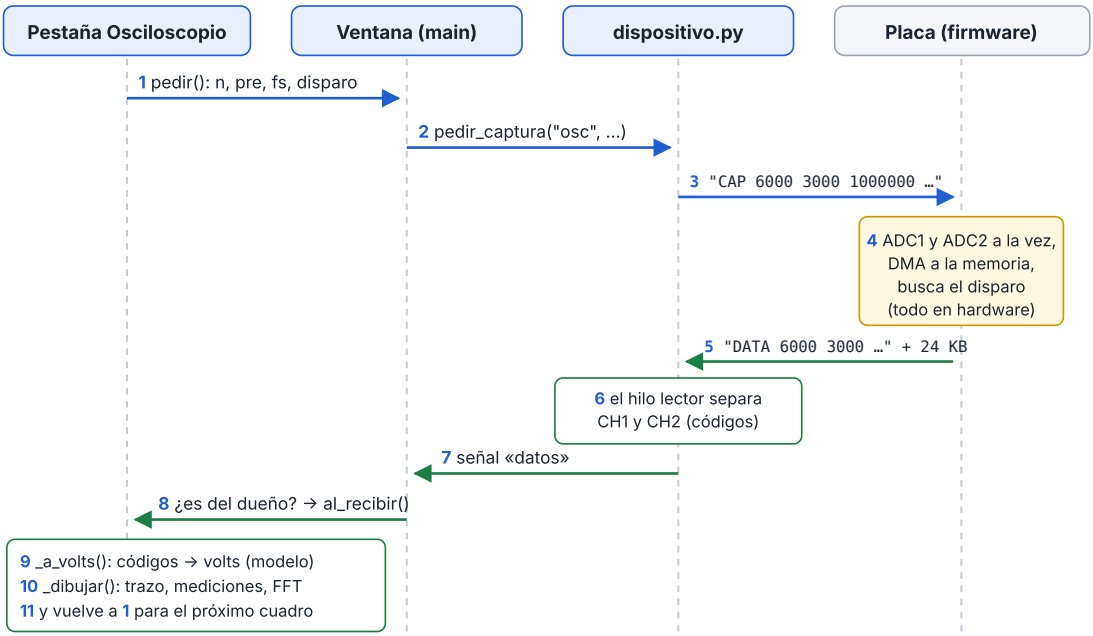
Hilo (thread). Leer el USB es esperar bytes que llegan cuando llegan. Si eso se hiciera en el mismo hilo que dibuja la ventana, la app se congelaría mientras espera. Por eso `dispositivo.py` lee en un **hilo aparte** y la ventana sigue respondiendo.

Señal de Qt. Es la forma segura de avisar algo de un hilo a otro. Cuando el hilo lector termina de juntar una captura, emite la señal `datos`; Qt se encarga de ejecutar en el hilo de la ventana la función conectada a esa señal (`Ventana._al_datos`).

El «dueño» de la captura. Varias partes piden capturas: el osciloscopio, el Bode, la calibración y el asistente del AD9833. La Ventana anota quién pidió la última (`pedir_captura`) y le entrega los datos solo a ese dueño. Además, si hay un barrido de Bode o una calibración en curso, el osciloscopio no puede arrancar RUN, Single ni Autoset (`captura_ocupada`); así nadie le roba la captura a otro.

3. Cómo viaja un dato

Ejemplo: un cuadro del osciloscopio en RUN. Se repite decenas de veces por segundo.



Azul: el pedido (PC → placa). Verde: la respuesta (placa → pantalla). Lo pesado, muestrear a 1 MS/s, lo hace el hardware del STM32; la PC recibe un bloque ya listo.

El mismo camino, en palabras

1. `tab_osciloscopio.parametros_captura()` mira la base de tiempo (por ejemplo 200 µs/div) y calcula cuántos puntos pedir, a qué frecuencia de muestreo y cuántos antes del disparo.
2. `main.Ventana.pedir_captura()` anota que el dueño es «osc» y manda el comando `CAP` por `dispositivo.enviar()`.
3. La placa captura y contesta `DATA n pre fs disparado` seguido de los datos: 4 bytes por muestra (CH1 en los 2 bytes bajos y CH2 en los 2 altos, tal como los deja el ADC del STM32).
4. El hilo de `dispositivo._lector()` junta los bytes, separa los dos canales con numpy y emite la señal `datos`.
5. `Ventana._al_datos()` comprueba que esa captura sea la que se pidió y se la pasa a la pestaña dueña.
6. `TabOsciloscopio._a_volts()` pasa los códigos (0 a 4095) a volts con la calibración (`modelo.Recta`); `_dibujar()` pinta lo visible y mide con `modelo.medir()`. Y se pide el próximo cuadro.

Y al revés: cuando cambiás algo en el Generador

`PanelGenerador` lee los controles y `modelo.armar_tabla()` arma un período de la forma en 4096 puntos (ya en códigos del DAC, con la calibración). `TabGenerador` espera 80 ms por si seguís girando la perilla y después manda **solo lo que cambió**: la tabla en binario (`GTBL`) y la frecuencia (`GSET`). La placa recorre esa tabla con el DAC y el DMA a la velocidad justa para dar la frecuencia pedida, y contesta la frecuencia real.

4. Archivo por archivo

Ordenados de lo más general a lo más específico. Cada archivo empieza con un comentario que explica lo mismo que acá, así que se puede seguir con el código abierto.

`main.py`

~580 líneas · arranque y ventana principal

El punto de entrada: `python main.py` abre la app.

- **Elegir la placa** al abrir (`DialogoPlaca`): F767ZI o F429ZI. Cada una guarda su propia configuración.
- **Ventana**: la barra de arriba (puerto, Conectar, ⚙ resolución) y las seis pestañas.
- **Reparte los mensajes** de la placa: líneas de texto al Generador y a Herramientas; capturas al dueño.
- **Reconexión automática**: si la placa se reinicia o se mueve el cable, busca el puerto 20 s y se reconecta sola.
- Compara la **versión del firmware** con la de la app y avisa si hay que cargar el firmware nuevo.

Si preguntan «¿dónde empieza todo?»: en la función `main()` al final de este archivo.

`dispositivo.py`

~230 líneas · comunicación USB

El único archivo que toca el puerto COM.

- `listar_puertos()` : pone primero el USB del Nucleo y marca el del ST-LINK como «no es este».
- `_conectar()` (en un hilo): abre el puerto, pregunta `ID?` (qué placa y versión) e `INFO?` (frecuencias máximas, tamaño de ventana...).
- `_lector()` (otro hilo): separa las líneas de texto y los bloques binarios de `DATA` .
- `enviar()` y `enviar_tabla()` : mandan comandos y la tabla del generador con una suma de control, para detectar bytes perdidos.

`modelo.py`

~1430 líneas · el «cerebro» matemático

Todas las cuentas, sin nada de interfaz. Es el más largo y el más probado.

- **Recta**: pasa un código del ADC o del DAC a volts y al revés. Una calibración es guardar dos puntos de esta recta.
- **Generador**: `forma_normalizada()` (seno, cuadrada, triangular, ruido, arbitraria desde CSV) y `armar_tabla()`.
- **Mediciones**: `medir()` da Vpp, máx, mín, media, RMS, frecuencia, período y ciclo útil.
- **Bode**: `fasor_seno()` ajusta una senoidal por mínimos cuadrados (amplitud y fase); `respuesta_bode()` calcula H = salida/entrada; `alias()`, `fs_submuestreo()` y `estimar_frecuencia()` permiten medir en MHz con el AD9833; `ajustar_bode()` encuentra polos y ceros.
- **FFT**: ventanas (Hann, Flat-top...), espectro en dB, piso de ruido, picos y armónicos (THD, SNR).
- **AD9833**: frecuencia y fase reales que puede dar el chip, amplitud en Vpp con la calibración, comandos.
- **Config**: la configuración de cada placa, guardada en un archivo JSON en la carpeta del usuario (`%APPDATA%\InstrumentoF767`).

Si preguntan «¿por qué separado?»: porque así cada cuenta se prueba sola (carpeta `tests`) y la interfaz se puede cambiar sin romper los cálculos.

`widgets.py`

~370 líneas · estilo y controles comunes

- Colores y hoja de estilo del tema oscuro; colores de CH1 (amarillo) y CH2 (celeste).
- Escala de la interfaz: `pt()` y `px()` agrandan o achican todo según la resolución elegida en ⚙.
- `EntradaSI`: campo que entiende «2k», «1,5 m» o «10 µ» y se mueve con la rueda del mouse.
- Atajos para armar filas, combos, grupos y botones con el mismo aspecto en todas las pestañas.

`tab_osciloscopio.py`

~1400 líneas · pestaña Osciloscopio

- **RUN / Single / Autoset**: el ciclo de pedir, recibir y dibujar explicado en la sección 3. Autoset prueba frecuencias de muestreo hasta ver al menos 3 períodos y después elige las escalas.
- **Disparo** por CH1, CH2 o libre, por flanco de subida o bajada, con nivel arrastrable en la pantalla.
- **Mediciones** de los dos canales, **cursores**, **modo XY** (figuras de Lissajous) y **suavizado** por promedio móvil.
- **Persistencia** (clase `Fosforo`): acumula los trazos y los va apagando, como el fósforo de un osciloscopio analógico.
- **FFT** y la ventana de análisis de espectro (`DialogoFFT`): tabla de picos, THD, SNR.
- Exportar a **CSV** o a **imagen**.

`tab_generador.py`

~390 líneas · pestaña Generador

- `PanelGenerador`: un panel por canal (forma, frecuencia, Vpp, offset, fase, AM/FM, barrido) con vista previa.
- `TabGenerador`: junta los cambios 80 ms y manda solo los comandos distintos a los que ya tiene la placa; incluye el panel del AD9833.

`panel_dds.py`

~800 líneas · módulo AD9833

- `PanelDDS`: frecuencia de 0,1 Hz a 12,5 MHz, forma, amplitud (potenciómetro digital), fase y modulaciones (barrido, FSK, PSK, ráfaga, AM) que hace el firmware.
- **Medir reloj...**: saca una cuadrada de 1 MHz, la mide con el frecuencímetro y corrige el error del cristal de 25 MHz del módulo.
- `DialogoCalDDS`: asistente de calibración de amplitud, paso a paso o automático con el CH1. Después se puede pedir la amplitud directo en Vpp.

`tab_bode.py`

~580 líneas · pestaña Bode

- Barrido logarítmico: para cada frecuencia pone el generador (interno o AD9833), captura CH1 y CH2 y calcula H = CH2/CH1.
- Si la salida es chica, **promedia** más capturas: promedia H como número complejo, así el ruido se cancela en vez de sumarse.
- Al terminar **ajusta un modelo** de polos y ceros (curva punteada), dice el tipo de filtro y la frecuencia de corte.
- Exporta a **CSV** y a **informe PDF**.

`informe_bode.py`

~550 líneas · informe PDF del Bode

- Dibuja el PDF con Qt (`QPdfWriter` + `QPainter`), midiendo todo en milímetros (clase `Documento`).
- Datos del ensayo, tabla de frecuencias características, H(s), gráficos medición contra modelo, plano s con polos y ceros, y anexo con todos los puntos.

`tab_calibracion.py`

~240 líneas · pestaña Calibración

- **Salidas**: la app pone dos códigos, medís con un multímetro y se guarda la recta real de cada canal.
- **Entradas** (automática): uniendo OUT con IN, el generador pone dos tensiones conocidas y se lee qué código da el ADC.

`tab_herramientas.py` ~145 líneas · Frecuencímetro y PWM

- Frecuencímetro (entrada D32): la placa elige sola entre contar flancos (frecuencias altas) o medir el período (bajas).
- PWM con frecuencia, ciclo útil y tiempo muerto entre las dos salidas complementarias.

`tab_placa.py` ~760 líneas · pestaña Placa

- Capacidades y límites de la placa elegida, datos en vivo de la placa conectada.
- Dibujo de la Nucleo con los pines que usa el instrumento (se resaltan al pasar el mouse) y esquemas de conexión de cada función.

`diag_puerto.py` ~20 líneas · diagnóstico

- Script suelto, fuera de la app: abre un puerto, pregunta `ID?` e `INFO?` y muestra todo lo que llega. Sirve para ver si la placa responde.

5. Los otros archivos del proyecto

Archivo o carpeta	Qué es
`tests/`	Pruebas automáticas (114). Se corren con `python -m unittest discover -s tests`. Prueban las cuentas de `modelo.py`, el Bode con circuitos simulados, el ajuste de polos y ceros, el informe PDF y partes de la interfaz.
`pruebas/nucleo_simulado.py`	Una **placa de mentira**: contesta los mismos comandos que el firmware y genera señales simuladas (por ejemplo un filtro RC). Permite probar la app entera sin tener la placa.
`pruebas/prueba_app.py`	Abre la app, la conecta a la placa simulada, toca botones solo y comprueba los resultados (frecuencia medida, Vpp, Bode del RC...).
`requirements.txt`	Las bibliotecas que hay que instalar: PySide6, pyqtgraph, numpy y pyserial.
`build_exe.bat` + `Instrumento.spec`	Arman `Instrumento.exe` con PyInstaller, para usar la app en una PC sin Python.
`firmware/`	El código C de las dos placas. Los archivos propios del instrumento son idénticos para la F767ZI y la F429ZI; las diferencias están en `placa.h`.
`hardware/`	Esquemas de las placas adicionales (salidas de ±5 V, entradas de ±10 V, frecuencímetro y fuente) y del módulo AD9833.
`CHANGELOG.md`	Las notas de cada versión.

6. Ideas para contar y preguntas probables

Cinco ideas que resumen el diseño

1. **Reparto del trabajo:** la placa hace lo que necesita tiempo real (muestrear, generar) y la PC todo lo que necesita cuentas o pantalla.
2. **Cálculo separado de la interfaz:** `modelo.py` no conoce la ventana, así que todo se puede probar automáticamente.
3. **La ventana nunca se congela:** el USB se lee en un hilo aparte y avisa con señales de Qt.
4. **Protocolo simple y robusto:** comandos de texto legibles, binario solo donde hace falta velocidad, sumas de control, reconexión automática.
5. **Probado sin hardware:** una placa simulada y más de cien pruebas automáticas permiten cambiar el código con confianza.

Preguntas que les pueden hacer

Pregunta	Respuesta corta
¿Por qué Python y no C# o LabVIEW?	Es gratis, corre en cualquier PC, numpy hace las cuentas rápido y Qt da una interfaz profesional. Con PyInstaller se arma un .exe.
¿Cómo sabe la app qué placa está conectada?	Le pregunta `ID?` y la placa contesta, por ejemplo, `INSTR-F767 v0.9.0`. Si no coincide con la elegida, ofrece cambiar.
¿Qué pasa si se pierden bytes en el USB?	Las tablas llevan una suma de control y la placa avisa si no coincide; si una captura no llega a tiempo, se vuelve a pedir; si el USB se corta, la app reconecta sola.
¿Cómo se convierte un código del ADC en volts?	Con una recta (dos puntos) que sale del diseño del circuito o, mejor, de la calibración. Está en `modelo.Recta`.
¿Cómo se refresca tan rápido?	La placa manda bloques ya capturados (decenas por segundo) y la app dibuja solo lo visible, sin antialiasing y reduciendo puntos (`curva_rapida`).
¿Dónde se guarda la configuración?	En un JSON por placa en la carpeta del usuario; se guarda sola 1,5 s después del último cambio.